

Exhaustive Search Project

Section:O42



Algorithms Design and Analysis, CS220 - PROJECT

Title:

Section:

The Students should
fill out these areas
"Yellow areas"

No.	Name	Academic number	Academic Email
1	Ghaleh Mubarak Albakheet	445009172	445009172@pnu.edu.sa
2	Layan Fayeze Ahmed	445009155	445009155@pnu.edu.sa
3	Reuf Ali Alghonaimi	445009194	445009194@pnu.edu.sa
4	Ajwan Abdullah Alomar	445009175	445009175@pnu.edu.sa

Computer Sciences Department

2024

Exhaustive Search Project

1. [Ghaleh] worked on writing the abstract (main idea), analyzed Problem 1 (finding elements in an array using the linear search algorithm), and wrote two references.
2. [Ajwan] wrote the introduction and analyzed Problem 1 (finding elements in an array using the linear search algorithm).
3. [Reuf] wrote the advantages, analyzed Problem 2 (the Traveling Salesman problem), and contributed one reference.
4. [Layan] analyzed Problem 2 (the Traveling Salesman problem), wrote the disadvantages.

Abstract

Our project is an exhaustive search. We started by explaining the concept of an exhaustive search. As we mentioned below, an exhaustive search's approach to search and check every element or solution within the problem domain until we find the optimal or correct solution. Since this algorithm is general, it solves many problems, and we have mentioned the most important and most common problems or applications that depend on it in solving them, such as finding the minimum element and Traveling salesman (TSP) . Followed by its advantages and disadvantages, with an explanation. Finally, at the analysis stage, we analyzed each problem separately in terms of time complexity with explanations, a pseudocode, steps, and the recurrence formula if one exists.

1- Introduction

Exhaustive search is a type of search that is guaranteed to find the optimal solution to a problem, it is also known as A brute force search. It is a straightforward process of solving a problem. Its approach is to search and check every element or solution among the problem domain such as lists, until find the optimal or correct solution. It is also based on trial and error where the programmer tries to merely utilize the computer's fast processing power to solve a problem, rather than applying some advanced algorithms and techniques developed with human intelligence.

Applications of exhaustive search:

- A simple example of applying exhaustive search would be linearly searching for an element in an array. When each element of an array is compared, such as finding the minimum value problem, it is the most direct and simple way for searching given data in the array.
- Another example of a problem that apply the brute-force approach is The Traveling Salesman Problem (TSP). It is a classic problem in computer science and operations research. It seeks to find the shortest possible route that visits each city exactly once and returns to the original city.

Advantages:

- It is efficient for solving small problems in scenarios where there are a small number of possibilities you need to work through, it may even be the most efficient option available.
- It is quite simple, does not require any domain knowledge since it does not use advanced techniques or any type of tricks.

Disadvantages:

- inefficient for large problems since finding the right option by this algorithm consumes a lot of time, the algorithm analysis goes above the $O(N)$ order of growth.

- This algorithm is neither constructive nor creative in comparison to algorithms that are constructed using some other design paradigms.
- Due to hardware limitations such as size of memory, the algorithm may fail to find out the solution of large-sized problems.

2. Analysis

In computer science, exhaustive search or brute-force search also known as generate and test, is a very general problem-solving technique and algorithmic paradigm that consists of systematically enumerating all candidates for the solution and checking whether each candidate satisfies the problem's statement. Thus, every different problem will have a different solving method, and those methods will be of different time complexities. We will discuss two different problems as follows:

Problem#1: Find element in array using linear search algorithm

```
findMin(arr)
  min = arr[0]
  For i = 1 to arr.length
    If arr[i] < min
      min = arr[i]
  Return min
```

Pseudocode of finding minimum element

How it works:

- Define an array of integers.
- Initialize a variable "min" to the first element of the array.
- Loop through the array and compare each element with the "min" variable.
- If an element is smaller than the "min" variable, update the "min" variable to that element.
- After the loop, print out the "min" variable with a message indicating that it is the minimum element in the array.

Time Complexity:

- Best Case : Occurs when there is only one element in the array, and the algorithm finds it immediately, taking only constant time. Therefore, the best-case time complexity is $T(n) = O(1)$.
- Worst Case : Occurs when the element being searched for is at the last position of the array. In this case, the algorithm will have to search through all n elements in the array before determining that the element is not present. Therefore, the worst-case time complexity is $T(n) = O(n)$.
- Average Case : Occurs when the element being searched for is randomly distributed throughout the array. In this case, there is no significant difference between the best and worst cases, and the algorithm takes $T(n) = O(n)$ time on average.

The recurrence equation:

$T(n) = T(n-1) + 1$ where $T(n)$ represents the time complexity of finding the minimum element in an array of size n using linear search. This is because, in each iteration of the for loop, the algorithm compares the current element to the current minimum element and updates the minimum if necessary.

This comparison takes constant time, so the time complexity of the algorithm is proportional to the number of iterations of the for loop. Since the for loop iterates $n-1$ times (starting from $i=1$),

we express the time complexity as a recurrence equation:

$$T(n) = T(n-1) + 1$$

The base case is $T(1) = 1$, since finding the minimum element in an array of size 1 takes constant time.

Proof using Substitution Method:

- Formula:
 - $T(n) = (n - 1) + 1$
 - Guess:
 - $O(n)$
 - Inequality:
 - $0 \leq f(n) \leq cg(n)$
 - $0 \leq T(n) \leq cn$
- Recurrence:
 - $T(n) \leq c(n-1) + 1$ Now, we will show that $T(n) \leq cn$:
- 1. Starting from the recurrence.
 - $T(n) \leq c(n-1) + 1$

2. Expanding the equation:
 - $T(n) \leq cn - c + 1$
3. Choosing c such that $-c + 1 \leq 0$:
 - This gives $c \geq 1$.
4. Therefore, we can write:
 - $T(n) \leq cn$
 - Thus, $T(n) = O(n)$ is correct with $c \geq 1$.

Implementation using python language:

```

project1.py > ...
1  def find_min(arr):
2      min_val = arr[0]
3      for i in range(1, len(arr)):
4          if arr[i] < min_val:
5              min_val = arr[i]
6      return min_val
7
8
9  numbers = [12, 7, 22, 5, 18]
10 min_element = find_min(numbers)
11 print("\nThe minimum element is:", min_element)

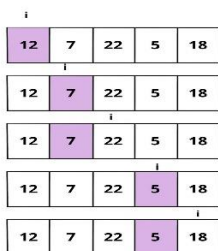
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\HUAWEI\myproject.py> & "C:/Program Files/Python

The minimum element is: 5

Simple Visualization:



Problem #2: Traveling salesman (TSP):

Given a set of cities and distances between them, find the shortest possible tour visiting each city exactly once.

```
Algorithm TSP-BruteForce(distance, N):
    minCost ← ∞ // Initialize minimum cost to a large value
    bestPath ← [] // Store the best path

    Function GeneratePermutations(path, cost):
        If all cities are visited:
            cost ← cost + distance[last city][start city] // Return to start
            If cost < minCost:
                minCost ← cost
                bestPath ← path.copy()
            Return

        For each unvisited city:
            Add city to path
            GeneratePermutations(path, cost + distance[last city][city])
            Remove city from path // Backtrack

    Start from city 0
    GeneratePermutations([0], 0)

    Return minCost, bestPath
```

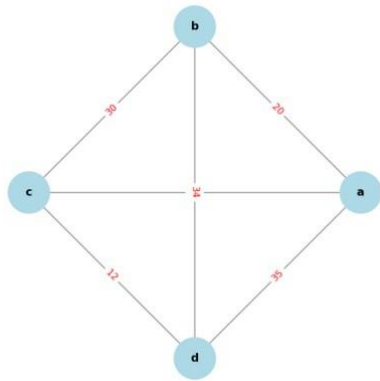
Pseudocode of TSP solved by brute-force Algorithm

How it works:

- Initialize minCost and bestPath.
- Generate all possible routes (permutations) recursively.
- Calculate the cost for each route.
- Update minCost if a better route is found.
- Use backtracking to explore all paths.

Problem-solving modeling

Weighted Graph Representation of TSP



TSP can be modeled as an undirected weighted graph. In the context of TSP, cities correspond to vertices, paths between cities correspond to edges, and the distance between cities (or the cost of travel) corresponds to the weight of the edge connecting the vertices.

Time complexity:

1. Worst Case Complexity $O(n!)$

In the worst case, we must explore every possible permutation of the cities.

Since we have $(n-1)!$ ways to arrange the remaining cities and checking each route takes $O(n)$ time (summing distances), Worst-case = $O(n \times (n-1)!) \approx O(n!)$

This occurs when no shortcuts or optimizations are found, and we must search through all possibilities.

2. Average Case Complexity $O(n!)$

The number of routes remains the same, and since every possible arrangement is checked in the same manner, the average case is also $O(n!)$.

No probabilistic advantage exists in brute force, so it performs equally poorly on average.

3. Best Case Complexity $O(n!)$

Even in the best case, we must generate all permutations to find the shortest path.

However, if an early solution is found, we can potentially terminate early, reducing execution time.

But in general, exhaustive search doesn't guarantee an early stop, so the best case remains $O(n!)$.

The Recurrence Equation :

$$T(n) = (n-1)T(n-1) + cn \text{ where}$$

- $T(n)$ is the time complexity of solving TSP for n cities.
- We try all $(n-1)$ possible next cities.
- Each recursive call reduces the problem to size $(n-1)$.
- The additional work done at each step is cn .

The base case is: $T(1) = c$

This is because if we solve TSP with only one city it will only takes constant time.

Proof Using the Substitution Method

$$T(n) \leq (n-1)T(n-1) + cn$$

Expand $T(n-1)$

$$T(n-1) \leq (n-2)T(n-2) + c(n-1)$$

Substitute back to the equation

$$T(n) \leq (n-1) [(n-2)T(n-2) + c(n-1)] + cn$$

Continuing the pattern we obtain

$$T(n) \leq (n-1)! T(1) + c \sum_{k=1}^{n-1} k(n-1)!$$

We know that $T(1) = c$, so

$$T(n) \leq (n-1)! c + c \sum_{k=1}^{n-1} k(n-1)!$$

Simplify

$$T(n) \leq c(n-1)! + c(n-1)! \frac{(n-1)n}{2}$$

$$T(n) \leq c(n-1)! \left(1 + \frac{n}{2}\right)$$

Approximate $(n-1)! \approx$

$$O(n!)$$

$$\left(1 + \frac{n}{2}\right)^n \text{ grows polynomially}$$

Therefore

$$T(n) = O(n!)$$

Implementation using python language

```
15 def calculate_cost(route):
16     total_cost = 0
17     n = len(route)
18     for i in range(n):
19         current_city = route[i]
20         next_city = route[(i + 1) % n] # Wrap around to the start of
            the route
21         # Look up the distance in both directions
22         if (current_city, next_city) in distances:
23             total_cost += distances[(current_city, next_city)]
24         else:
25             total_cost += distances[(next_city, current_city)]
26     return total_cost
27
28 # Generate all permutations of the cities
29 all_permutations = itertools.permutations(cities)
30
31 # Initialize variables to track the minimum cost and corresponding
    route
32 min_cost = float('inf')
33 optimal_route = None
34
35 # Iterate over all permutations and calculate costs
36 for perm in all_permutations:
37     cost = calculate_cost(perm)
38     if cost < min_cost:
39         min_cost = cost
40         optimal_route = perm + ('a',)
41
42 # Print the optimal route and its cost
43 print(f"Optimal Route: {optimal_route}")
44 print(f"Total Cost: {min_cost}")
```

```
Optimal Route: ('a', 'b', 'c', 'd', 'a')
Total Cost: 97
```

```
=== Code Execution Successful ===
```

3. References:

- [1] E.Horowitz, S.Sahni, S.Rajasekaran, Fundamentals of Computer Algorithms, Galgotia Publications. 20 May 2023 login.
- [2] T.H. Cormen, C.E. Leiserson, R.L.Rivest, C.Stein, Introduction to Algorithms, PHI. 19 May 2023 login.
- [3] Robert Sedgewick, Kevin Wayne, Algorithms, 4th edition, Pearson education, Inc, 2011.

4. Appendix:

Problem#1

//find the minimum element in the array

```
def find_min(arr):  
    min_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] < min_val:  
            min_val = arr[i]  
    return min_val
```

```
numbers = [12, 7, 22, 5, 18]  
min_element = find_min(numbers)  
print("\nThe minimum element is:", min_element)
```

Problem#2

// find the shortest possible tour visiting each city exactly once.

```
def calculate_cost(route):
    total_cost = 0
    n = len(route)
    for i in range(n):
        current_city = route[i]
        next_city = route[(i + 1) % n] # Wrap around
        to the start of the route
        # Look up the distance in both directions
        if (current_city, next_city) in distances:
            total_cost += distances[(current_city, next_city)]
        else:
            total_cost += distances[(next_city,
            current_city)]
    return total_cost

# Generate all permutations of the cities
all_permutations = itertools.permutations(cities)
# Initialize variables to track the minimum cost and
corresponding route
min_cost = float('inf')
optimal_route = None

# Iterate over all permutations and calculate costs
for perm in all_permutations:
    cost = calculate_cost(perm)
    if cost < min_cost:
        min_cost = cost
    optimal_route = perm + ('a',)
# Print the optimal route and its cost
print(f"Optimal Route: {optimal_route}")
print(f"Total Cost: {min_cost}")
```

